



Lyceum of the Philippines University – Cavite
College of Computer Studies

DCSN03C – Computer Programming 2

FINALS PROJECT GRADING RUBRIC

Text-Based RPG Game in Python

Academic Year 2025–2026 | Finals Period

1. Object-Oriented Design (25 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
Encapsulation	All attributes use <code>_private</code> naming with <code>@property</code> decorators; no direct access from outside.	Most attributes private; minor gaps in property usage.	Some attributes private; inconsistent conventions.	All attributes public; no use of <code>_private</code> or properties.	5
Inheritance	Clear hierarchy using <code>super().__init__()</code> ; at least 2 subclasses of Character.	Inheritance present but <code>super()</code> usage has minor issues.	One subclass exists; shallow hierarchy.	No inheritance used.	5
Polymorphism	Methods overridden meaningfully (e.g., <code>Player.attack()</code> vs <code>Enemy.attack()</code> behave differently).	Override present but differences are minimal.	One trivial override.	No method overriding.	5
Abstraction	<code>abc.ABC</code> used with <code>@abstractmethod</code> ; subclasses forced to implement key methods.	<code>ABC</code> exists but not all relevant methods are abstract.	Base class is concrete; serves as informal template.	No abstraction; standalone classes.	5
Class Design	5+ well-organized classes in separate <code>.py</code> files; single responsibility; clean imports.	5 classes but some modules do too much.	3–4 classes; everything in one file.	Fewer than 3 classes; procedural code.	5

2. Core Programming Constructs (25 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
Variables & Data Types	Correct use of <code>int</code> , <code>str</code> , <code>bool</code> , <code>list</code> , <code>dict</code> ; <code>UPPER_CASE</code> constants; meaningful names.	Mostly correct types; minor naming issues.	Some type mismatches; inconsistent naming.	Wrong types; meaningless variable names.	5
Conditional Statements	<code>if-elif-else</code> used for menus, combat, win/lose, and validation; clean logic.	Conditionals in most areas; minor logic errors.	Basic <code>if-else</code> only; no <code>elif</code> chains.	Minimal or broken conditional logic.	5
Iterative Statements	<code>while</code> loops for game/combat; for loops with <code>enumerate()</code> for inventory; input retry loops.	Loops present for most areas; one type missing.	Only <code>while</code> or only <code>for</code> used; limited iteration.	Loops absent or infinite loop issues.	5

Input Handling	try-except ValueError everywhere; while True loops for retries; no crashes.	Most input validated; minor edge cases.	Some validation; crashes on certain inputs.	No validation; crashes on bad input.	5
Output Formatting	Clean f-string output; aligned menus; HP bars or status displays; narrative text.	Readable output; minor formatting issues.	Output works but is cluttered.	Raw print() calls; hard to follow.	5

3. Game Functionality & Completeness (20 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
Character System	3+ distinct classes with different stats/abilities; smooth creation flow.	3 classes but minimal differentiation.	2 classes or classes feel identical.	1 or no class selection.	5
Combat System	Turn-based with attack, defend, items; random damage; balanced and engaging.	Combat works but lacks variety.	Basic attack-only combat.	Combat broken or non-interactive.	5
Progression	Leveling, EXP, stat growth, multiple stages leading to a boss.	Leveling works; limited stages.	Flat progression; no stat change.	No progression system.	5
Completeness	Win/lose conditions, replay via while loop, all features end-to-end.	Playable but one feature incomplete.	Multiple features partial.	Game not in a playable state.	5

4. Code Quality & Best Practices (10 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
PEP 8 & Readability	Consistent snake_case for functions/vars, PascalCase for classes; clean indentation; type hints used.	Mostly PEP 8 compliant; a few style issues.	Inconsistent style; some unreadable blocks.	No convention; single-letter variables.	5
Comments & Structure	Docstrings on all classes/methods; inline comments on complex logic; separate .py modules.	Some docstrings; structure is okay.	Minimal comments; everything in one file.	No comments; monolithic script.	5

5. Project Documentation (10 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
Content & Completeness	All 13 sections present and thorough; UML class diagram, flowcharts, and test cases included.	Most sections complete; minor omissions.	Several sections missing or superficial.	Documentation is a stub or absent.	5
Presentation & Format	Professional formatting; consistent layout; proper citations; clean design.	Good formatting; minor inconsistencies.	Readable but poorly organized.	Unformatted; no structure.	5

6. Presentation / Defense (10 Points)

Criteria	Excellent (5)	Good (4)	Satisfactory (3)	Needs Improvement (1-2)	Wt
Demo & Explanation	Live demo via python main.py runs flawlessly; explains OOP, code logic, and design decisions.	Demo works with minor issues; adequate explanation.	Demo partially works; struggles to explain code.	Demo fails; cannot explain own code.	5
Q&A / Understanding	Answers all questions confidently; explains Python-specific concepts	Answers most questions correctly.	Answers some; surface understanding.	Cannot answer basic questions.	5



(properties, ABC,
super()).

Score Sheet

Student/Group: _____ Section: _____ Date: _____

Category	Max Points	Score	Remarks
1. Object-Oriented Design	25		
2. Core Programming Constructs	25		
3. Game Functionality & Completeness	20		
4. Code Quality & PEP 8	10		
5. Project Documentation	10		
6. Presentation / Defense	10		
Bonus: JSON Save/Load (json module)	+5		
TOTAL	100		

Evaluated by: _____ Signature: _____